

RUDP — Explained Like You're Five

What this project actually does, why it does it, and how to run it — no computer science degree required.

What did we even build?

We built our own version of one of the systems that makes the internet work. Every time you load a webpage, send a message, or stream a video, your computer is chopping that data into little pieces called **packets** and sending them across the internet to another computer.

THINK OF IT LIKE THIS

Imagine you're mailing someone a 500-page book, but your mailbox can only fit one page per envelope. You'd need to number every page, mail them all separately, and hope your friend can put them back in order — and re-mail any page that gets lost. That's basically what happens every time you use the internet.

There are two common ways to send these packets:

UDP — the "fire and forget" method. Fast, but if a page gets lost in the mail, nobody notices or resends it. Great for things like video calls where a tiny glitch doesn't matter, bad for anything that needs to be perfect.

TCP — the "reliable" method your web browser actually uses. It numbers every page, checks that all of them arrived, and automatically re-sends anything that got lost — so you always get the complete, correct file.

We built our own mini version of that reliable system — called **RUDP** (Reliable UDP) — completely from scratch, in the C programming language. We didn't use any of the internet's built-in reliability tools. We built the "make sure nothing gets lost" logic ourselves.

Why does that matter?

Anyone can use TCP without knowing how it works — like anyone can drive a car without knowing how an engine works. This project is us building the engine. It proves we understand *why* the internet doesn't lose your data, not just that it doesn't.

How does it actually work? (the simple version)

1. Saying hello first

Before sending any real data, our two programs (a "client" and a "server") shake hands — literally exchange three small messages to agree "yes, we're both here and ready." This is called a **handshake**.

2. Numbering everything

Every piece of data gets a number, like numbering pages in that book analogy. If a page arrives out of order, the receiver knows exactly where it goes.

3. Checking everything arrived

The receiver sends back a little "got it!" message (called an **ACK**, short for acknowledgment) for every piece it receives. If the sender doesn't hear "got it!" within a short time, it assumes that piece got lost and sends it again automatically.

4. Not overwhelming the network

Here's the clever part — our program doesn't just blast everything at once. It starts sending a few pieces at a time, and if everything arrives fine, it sends a few more next time, gradually speeding up. But the moment something gets lost, it slows back down immediately — like a driver easing off the gas when they see traffic ahead. This is called **congestion control**, and it's exactly what real internet traffic does to avoid jamming everything up.

THINK OF IT LIKE THIS

It's like carrying boxes up a staircase. You start with one box to test the stairs aren't broken. They're fine, so next trip you carry two. Then four. Then eight — getting faster and faster. But the moment you trip and drop a box, you don't keep grabbing more boxes recklessly — you carry fewer for a while until you're confident again.

Did it actually work?

Yes — and we proved it, not just claimed it. We deliberately made packets get lost on purpose (up to 40% of them — meaning almost half the "pages" vanished in the mail) and our program still delivered the complete, perfect message every single time. We even watched the raw network traffic with a tool called Wireshark to independently confirm it was really happening, not just something our own program was telling us.

How to actually run it

You'll need two windows open side by side — think of one as "the person receiving mail" and the other as "the person sending mail."

- 1 **Open a terminal (a text-based window where you type commands) and go to the project folder**

```
cd ~/projects/rudp
```

- 2 **Build the program (this turns our code into something the computer**

can actually run)

```
make
```

3 In that first window, start the "receiver"

```
./bin/server
```

This window will look "stuck" — that's normal, it's just waiting for someone to talk to it. Leave it open.

4 Open a second terminal window, go to the same folder, and start the "sender"

```
cd ~/projects/rudp  
./bin/client
```

5 Watch it work

You'll see messages flying by showing packets being sent, acknowledged, and the "sending speed" (called `cwnd`) going up and down. When it says

"All packets delivered"

at the end, it worked — the whole message arrived perfectly.

6 Want to see it survive chaos? Try this instead of step 4

```
./bin/client 20
```

The "20" tells it to randomly throw away 20% of the packets on purpose — you'll see it detect the loss and automatically fix itself.

Quick answers to obvious questions

Do I need to understand the code to run it?

No. Just follow the steps above exactly. The program does all the thinking.

What if a window says "command not found" or shows an error?

Make sure you actually typed `cd ~/projects/rudp` first in that specific window — commands only work if you're standing in the right folder.

How do I stop it?

Click into the window and press **Ctrl and C at the same time**. That stops whatever's running in that window.

Is this dangerous or going to break my computer?

No — it only talks to itself on your own machine (nothing goes out onto the real internet), and it can't damage anything.

